

SIMATS ENGINEERING

CO2- ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1103

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 60%

QUESTIONS: 6

Total Marks:60

Annexure A

Application Based Questions

Code of Conduct:

*I **GIRIJA B/192311044** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIRIJA B

Annexure A

Application Based Questions

1. Use Case Diagram Analysis

A Smart Hotel Reservation System allows guests to search rooms, make reservations, cancel bookings, and process payments. Managers handle room allocation and reports, while payment services process transactions. The current design does not clearly distinguish actor responsibilities.

Analyze the system requirements, identify missing actors and use cases, and design an optimized Use Case Diagram showing proper relationships (include, extend, generalization) for better system clarity.

(a) Analysis of the System Requirements

The Smart Hotel Reservation System enables customers to book hotel rooms online while allowing hotel staff to manage reservations efficiently.

Actors Identified

1. Guest (Primary Actor) – Searches rooms, books rooms, cancels bookings, makes payments.
2. Manager (Primary Actor) – Allocates rooms, manages bookings, generates reports.
3. Payment Service (External Actor) – Processes online payments.
4. Receptionist (Missing Actor) – Confirms bookings, checks guests in/out.
5. Administrator (Missing Actor) – Manages users, rooms, pricing, and system maintenance.

(b) Missing Actors and Use Cases

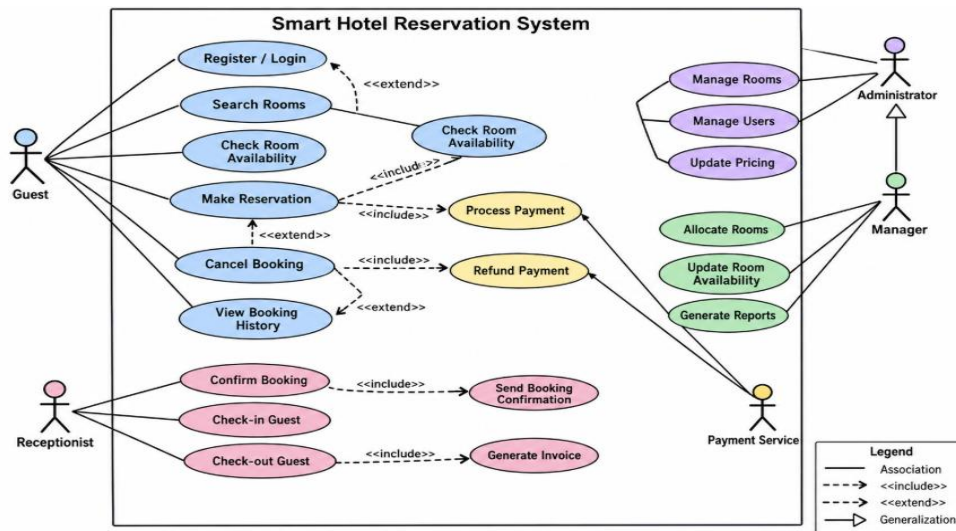
Missing Actors

- Receptionist
- Administrator

Missing Use Cases

- Register/Login
- Check Room Availability
- View Booking History
- Confirm Booking
- Check-in Guest
- Check-out Guest
- Generate Invoice
- Manage Rooms
- Manage Users
- Update Room Availability

- Refund Payment
- Send Booking Confirmation



2. Class Diagram Design

A University Management System includes entities such as Student, Faculty, Course, Department, and Examination. The existing design lacks proper attributes, operations, and relationships among classes.

Analyze the system, identify relevant classes with attributes and methods, define relationships (association, aggregation, inheritance), and construct a well-structured Class Diagram.

(a) Analysis of the System

A **University Management System** is used to manage students, faculty members, departments, courses, and examinations. The existing design is incomplete because it lacks proper class attributes, operations (methods), and relationships. An optimized class diagram should clearly represent the data, functions, and interactions among all classes.

(b) Relevant Classes with Attributes and Methods

1. Person (Superclass)

Attributes: PersonID : int, Name : String, Email : String

Methods: Login(), Logout()

2. Student (Subclass of Person)

Attributes: StudentID : int, Semester : int, CGPA : float

Methods: EnrollCourse(), ViewResult(), PayFees()

3. Faculty (Subclass of Person)

Attributes: FacultyID : int, Designation : String, Salary : double

Methods: TeachCourse(), EvaluateExam(), UploadMarks()

4. Department

Attributes: DeptID : int, DeptName : String

Methods: AddCourse(), AssignFaculty()

5. Course

Attributes: CourseID : int, CourseName : String, Credits : int

Methods: AssignStudent(), ScheduleClass()

6. Examination

Attributes: ExamID : int, ExamDate : Date, TotalMarks : int

Methods: ConductExam(), PublishResult()

(c) Relationships Among Classes

1. Inheritance (Generalization)

- Student inherits Person.
- Faculty inherits Person.

2. Association

- Student enrolls in Course (Many-to-Many).
- Faculty teaches Course (One-to-Many).
- Student appears for Examination (Many-to-Many).
- Course has Examination (One-to-Many).
- Department is associated with Faculty.

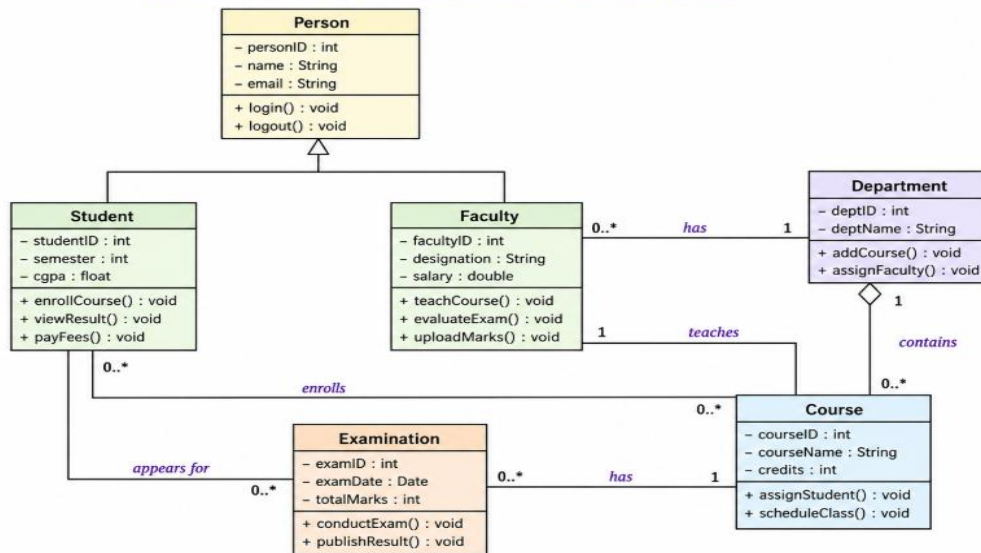
3. Aggregation

- One Department contains multiple Courses.

Department ◇———— Course

The Course can exist independently of the Department; therefore, Aggregation is used.

University Management System – Class Diagram



3. Sequence Diagram Evaluation

In a Vehicle Rental System, a customer searches for a vehicle, books it, makes payment, and receives a rental confirmation. The interaction among system components is incomplete and poorly documented.

Analyze the process flow, identify missing objects and message sequences, and design a detailed Sequence Diagram representing proper interaction among system components.

(a) Analysis of the Process Flow

In a Vehicle Rental System, a customer performs the following activities:

- Searches for available vehicles.
- Selects a vehicle and enters rental details.
- Makes payment for the booking.
- Receives rental confirmation.

The existing interaction is incomplete because it does not clearly show how the system checks availability, processes payment, stores booking information, and sends confirmation.

(b) Missing Objects Identified

The original design only considers the customer and booking process. The following objects are required for a complete sequence diagram:

Object	Purpose
Vehicle Inventory	Stores available vehicle information.

Object	Purpose
Booking Service	Handles booking logic and rental calculations.
Payment Gateway	Processes online payments securely.
Notification Service	Sends confirmation through Email/SMS.
Database	Stores booking and payment records.

(c) Objects (Lifelines) in the Sequence Diagram

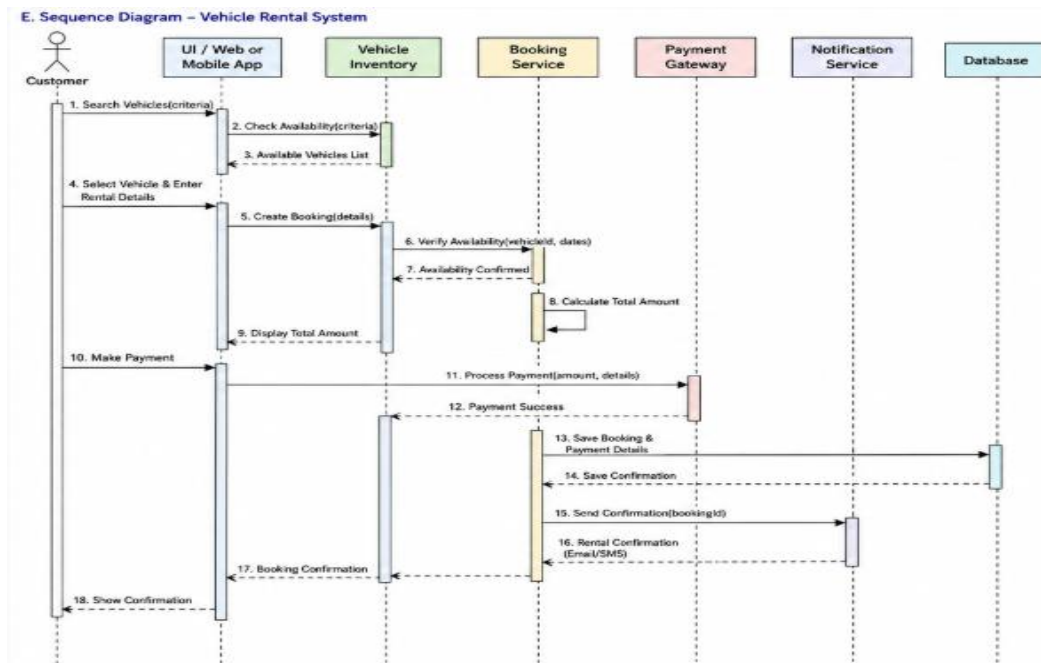
The optimized sequence diagram contains the following lifelines:

- Customer – Actor who rents the vehicle.
- UI / Web or Mobile App – User interface for searching and booking.
- Vehicle Inventory – Checks vehicle availability.
- Booking Service – Manages booking workflow.
- Payment Gateway – External payment processing service.
- Notification Service – Sends rental confirmation.
- Database – Stores booking and payment details.

(d) Detailed Message Sequence

The correct interaction among system components is shown below.

- Customer → UI: Search Vehicles(criteria)
- UI → Vehicle Inventory: Check Availability(criteria)
- Vehicle Inventory → UI: Available Vehicles List
- Customer → UI: Select Vehicle & Enter Rental Details
- UI → Booking Service: Create Booking(details)
- Booking Service → Vehicle Inventory: Verify Availability(vehicleId, dates)
- Vehicle Inventory → Booking Service: Availability Confirmed
- Booking Service → Booking Service: Calculate Total Amount
- Booking Service → UI: Display Total Amount
- Customer → UI: Make Payment
- UI → Payment Gateway: Process Payment(amount, details)
- Payment Gateway → Booking Service: Payment Success
- Booking Service → Database: Save Booking & Payment Details
- Database → Booking Service: Save Confirmation
- Booking Service → Notification Service: Send Confirmation(bookingId)
- Notification Service → Customer: Rental Confirmation (Email/SMS)
- Booking Service → UI: Booking Confirmation
- UI → Customer: Show Confirmation



4. State Diagram Construction

A Traffic Signal Control System operates through states such as Red Signal, Green Signal, Yellow Signal, Emergency Mode, and Maintenance Mode. The transitions between states are not clearly defined.

Analyze the system behavior, identify valid states and transitions, and construct a State Transition Diagram ensuring all events and conditions are properly represented.

(a) Analysis of the System Behavior

A **Traffic Signal Control System** controls traffic flow by changing signal lights in a predefined sequence. Under normal operation, the system cycles through **Red**, **Green**, and **Yellow** signals. In special situations, such as emergency vehicle detection or maintenance work, the system enters **Emergency Mode** or **Maintenance Mode**. The existing design does not clearly define the transitions between these states, making the system behavior ambiguous.

(b) Valid States Identified

The Traffic Signal Control System consists of the following states:

State	Description
Initial State	System starts operation.
Red Signal	Vehicles must stop.
Green Signal	Vehicles are allowed to move.
Yellow Signal	Warns vehicles that the signal will change.

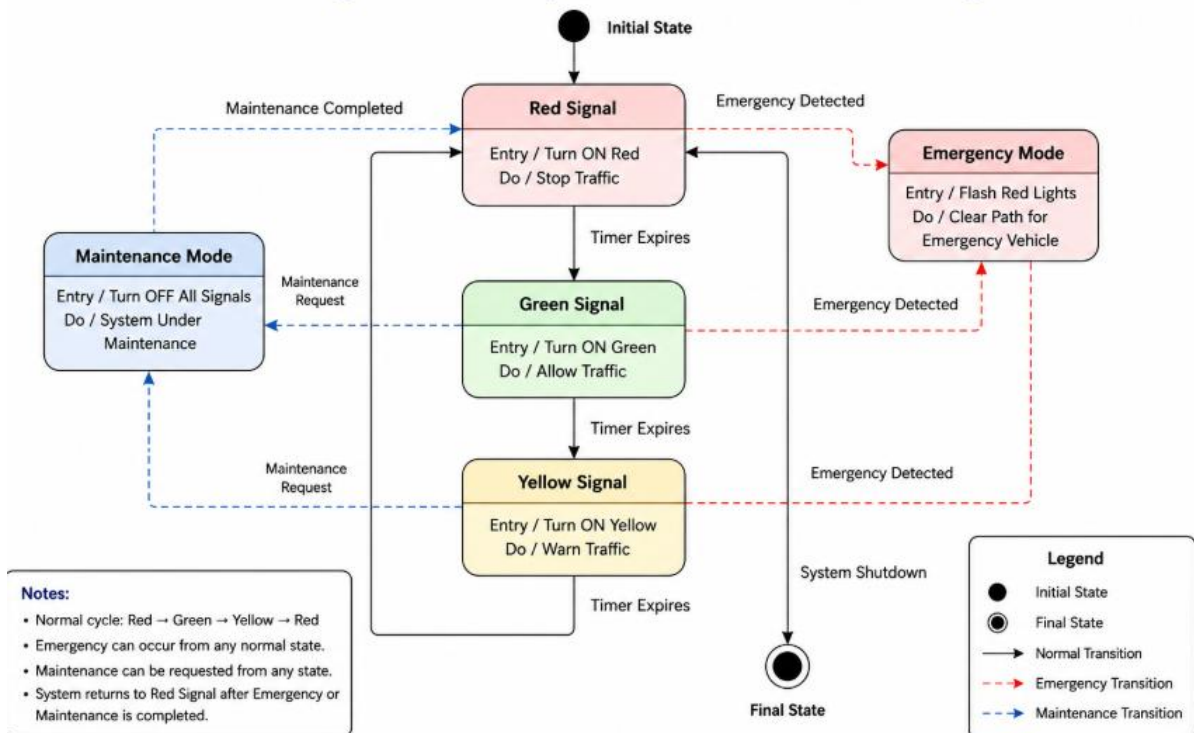
State	Description
Emergency Mode	Activated for emergency vehicles or emergency situations.
Maintenance Mode	Activated during maintenance or system repair.
Final State	System shutdown.

(c) Valid State Transitions

The optimized state transitions are:

- **Initial State → Red Signal**
- **Red Signal → Green Signal** (Timer expires)
- **Green Signal → Yellow Signal** (Green timer expires)
- **Yellow Signal → Red Signal** (Yellow timer expires)
- **Any Normal State → Emergency Mode** (Emergency vehicle detected)
- **Emergency Mode → Red Signal** (Emergency cleared)
- **Any State → Maintenance Mode** (Maintenance request)
- **Maintenance Mode → Red Signal** (Maintenance completed)
- **Red Signal → Final State** (System shutdown)

Traffic Signal Control System – State Transition Diagram



5. Component Diagram Design

A Smart Healthcare Portal consists of modules such as Patient Management, Appointment Scheduling, Medical Records, Billing, and Notification Services. The current architecture does not clearly define component interactions and dependencies.

Analyze the system architecture, identify major components and their interactions, and design a Component Diagram showing dependencies and interfaces between modules.

(a) Analysis of the System Architecture

A **Smart Healthcare Portal** provides online healthcare services by integrating different functional modules. Patients can manage their profiles, schedule appointments, access medical records, make bill payments, and receive notifications. The existing architecture does not clearly define the interactions and dependencies among these modules. An optimized Component Diagram should identify the major software components, define their interfaces, and illustrate how they communicate with one another.

(b) Major Components Identified

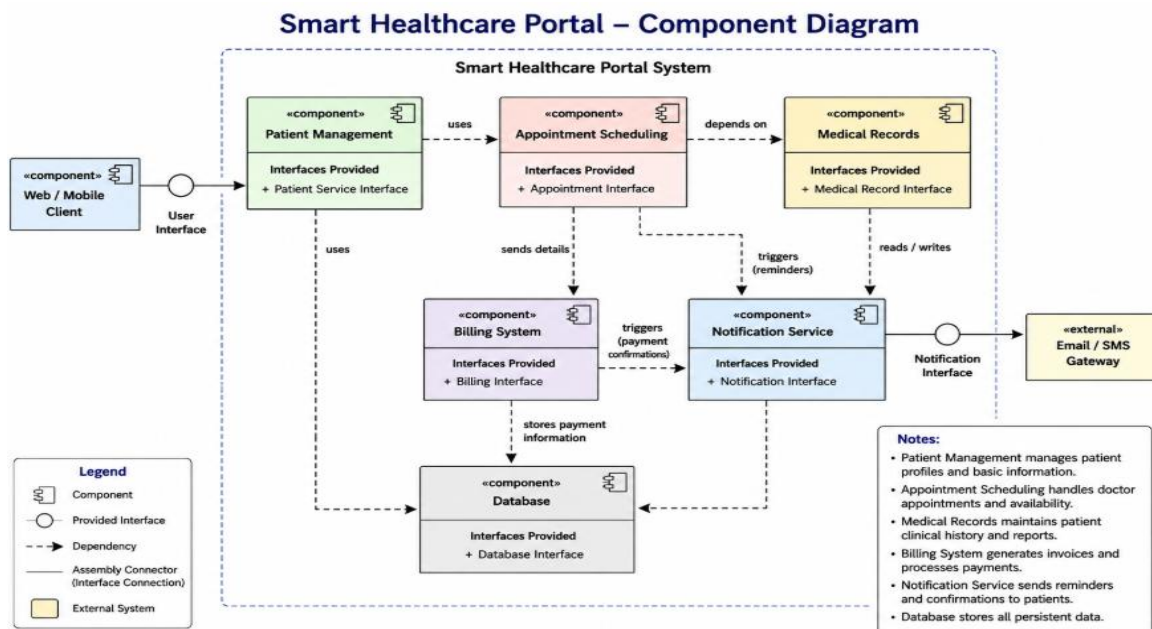
The Smart Healthcare Portal consists of the following components:

Component	Purpose
Patient Management	Stores patient information and profile details.
Appointment Scheduling	Manages doctor appointments and schedules.
Medical Records	Maintains patient medical history and reports.
Billing System	Generates invoices and processes payments.
Notification Service	Sends appointment reminders and payment confirmations.
Database	Stores patient, appointment, billing, and medical record data.

(c) Component Interactions and Dependencies

The interactions among the components are as follows:

- **Patient Management** communicates with **Appointment Scheduling** to schedule appointments.
- **Appointment Scheduling** accesses **Medical Records** to retrieve patient information.
- **Medical Records** interacts with the **Database** to store and retrieve records.
- **Billing System** receives appointment details and generates invoices.
- **Billing System** stores payment information in the **Database**.
- **Notification Service** receives information from Appointment Scheduling and Billing System to notify patients.
- All major components access the **Database** for persistent storage.



6. Deployment Diagram Design

A Smart City Waste Management System is deployed across IoT waste-bin sensors, edge devices, cloud servers, and monitoring dashboards. The current design lacks clarity regarding physical architecture and communication links.

Analyze the deployment environment, identify nodes, artifacts, and communication links, and construct a detailed Deployment Diagram representing the complete system architecture.

(a) Analysis of the Deployment Environment

A **Smart City Waste Management System** uses Internet of Things (IoT) technology to monitor waste bins distributed throughout a city. Sensors installed in the bins collect fill-level data and send it to nearby edge devices. The edge devices preprocess the data and forward it to cloud servers for storage and analysis. The processed information is displayed on monitoring dashboards used by municipal authorities.

The current deployment architecture does not clearly identify the physical nodes, deployed software artifacts, and communication links between them. An optimized Deployment Diagram should clearly represent the physical infrastructure and communication flow.

(b) Nodes and Artifacts Identified

Physical Nodes

Node	Description
IoT Waste-bin Sensor	Measures waste level and bin status.

Node	Description
Edge Device / Gateway	Collects sensor data and performs preprocessing.
Cloud Server	Hosts application services and analytics.
Database Server	Stores waste management data.
Monitoring Dashboard	Displays reports and alerts to city officials.
Administrator PC / Mobile	Used by municipal authorities to monitor the system.

Artifacts Deployed

Node	Artifact
IoT Waste-bin Sensor	Sensor Firmware
Edge Device	Edge Processing Software
Cloud Server	Waste Management Application
Database Server	Waste Management Database
Monitoring Dashboard	Web Dashboard
Administrator Device	Dashboard Client

(c) Communication Links

The communication between nodes is as follows:

- **IoT Waste-bin Sensors → Edge Device** using **Wi-Fi / LoRa / ZigBee**
- **Edge Device → Cloud Server** through the **Internet**
- **Cloud Server → Database Server** for data storage and retrieval
- **Cloud Server → Monitoring Dashboard** for visualization and reporting
- **Administrator Device → Monitoring Dashboard** through a web browser
- **Monitoring Dashboard ↔ Cloud Server** for real-time updates

Smart City Waste Management System – Deployment Diagram

